



Herança

- `extends` para criar uma subclasse.
- Princípio do `é um` para criar uma subclasse.

Exemplo:

```
class Animal { //super classe
    void comer() {
    }
}

class Cachorro extends Animal { // subclasse
    void latir() {
    }
}
```

Neste exemplo, `Cachorro` é uma subclasse de `Animal`. Isso significa que um objeto `Cachorro` pode usar o método `comer()` da classe `Animal`, além do método `latir()` que é específico da classe `Cachorro`.



Sobrescrita de Métodos

Exemplo:

```
public class Animal {  
    public void fazerSom() {  
        System.out.println("O animal faz um som");  
    }  
}  
  
public class Cachorro extends Animal {  
    public void fazerSom() {  
        System.out.println("O cachorro late");  
    }  
}
```

Neste exemplo, a classe `Cachorro` sobreescreve o método `fazerSom()` da classe `Animal`. Portanto, quando você chama `fazerSom()` em um objeto `Cachorro`, a mensagem "O cachorro late" é exibida em vez de "O animal faz um som".



A Palavra-chave super

- referência à superclasse imediata.

Exemplo:

```
public class Animal {  
    public void fazerSom() {  
        System.out.println("O animal faz um som");  
    }  
}  
  
public class Cachorro extends Animal {  
    @Override  
    public void fazerSom() {  
        super.fazerSom();  
        System.out.println("O cachorro late");  
    }  
}
```

Neste exemplo, a classe `Cachorro` chama o método `fazerSom()` da superclasse `Animal` usando a palavra-chave `super`. Portanto, quando você chama `fazerSom()` em um objeto `Cachorro`, as mensagens "O animal faz um som" e "O cachorro late" são exibidas.